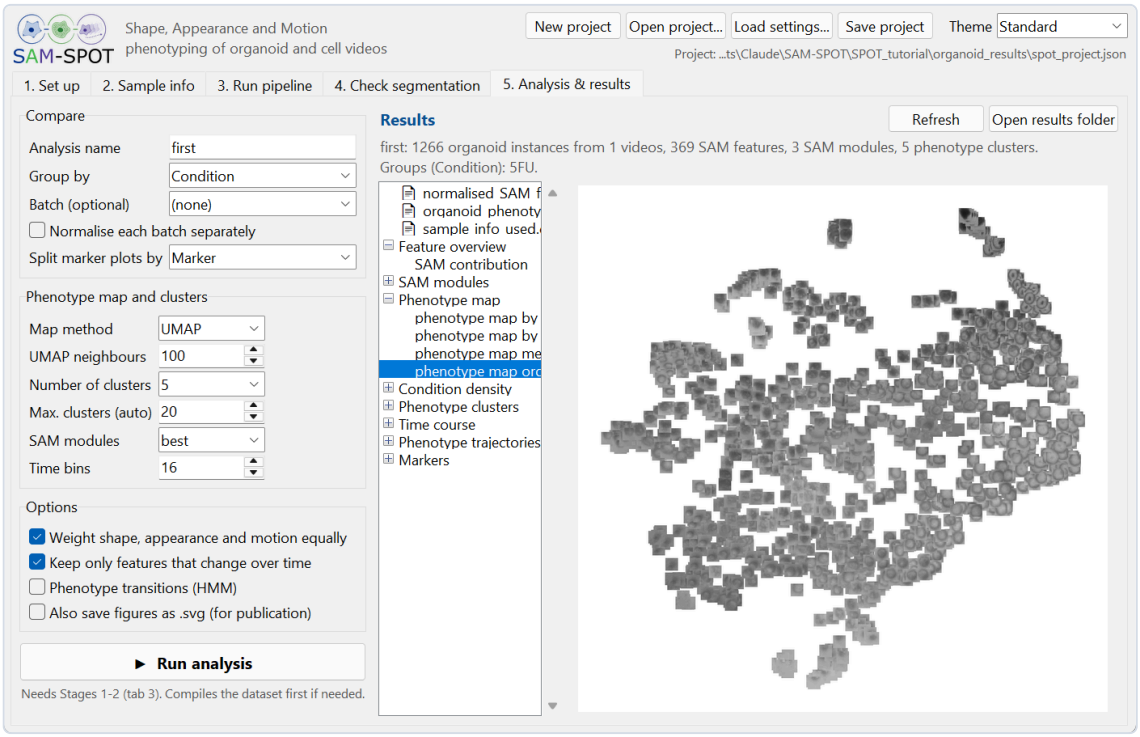
SAM-SPOT

SAM-SPOT graphical interface

Step-by-step tutorial: phenotyping organoid videos and Cell Tracking Challenge datasets without writing code



SAM-SPOT GUI 1.0 with SPOT 2.0 · gui/spot_gui.py

Contents

1. What the GUI does
2. Installing and starting the GUI
3. The window at a glance
4. Tutorial A: organoid videos
5. Tutorial B: a Cell Tracking Challenge dataset
6. Reading the results
7. Reproducibility, resuming and the command line
8. Themes
9. Troubleshooting

1. What the GUI does

SPOT (Shape, Appearance and Motion Phenotype Observation Tool) measures the **SAM phenome** of every organoid or cell in a time-lapse video: the shape, appearance and motion features of each object at each time point. It then maps all objects onto a phenotype landscape, finds SAM modules (groups of features that change together), phenotype clusters, the average phenotype trajectory of each condition and the transitions between phenotypes.

The GUI runs this whole workflow from a folder of videos to the figures, in three stages:

Stage	Steps
1. Find objects	Prepare the videos (grayscale, resize, drift correction) → detect organoids (YOLOv3 network) → track them over time → outline them (attention U-Net) → clean up the tracks. For a Cell Tracking Challenge dataset the annotations replace detection, tracking and segmentation.
2. SAM phenome	Shape, appearance and motion features of every object at every time point; optional fluorescent marker measurements; compile all videos into dataset tables.
3. Analysis	Feature normalisation, SAM modules, phenotype map (UMAP / PaCMAP), clusters, time course, trajectories, phenotype transitions (hidden Markov model) and marker plots.

The heavy computation runs in a separate process, so the window stays responsive, a run can be stopped at any time, and a stopped run resumes where it stopped.

2. Installing and starting the GUI

1. Install SPOT and its dependencies as described in the main `README.md` of the repository (a conda environment, e.g. `SAM-SPOT_env`). A GPU is used if available but is not required.
2. Start the GUI from the repository folder:

   ```
   conda activate SAM-SPOT_env
   python gui/spot_gui.py
   ```

   On Windows you can instead double-click `gui/launch_spot_gui.bat` (edit `SPOT_ENV` in it if your environment has another name).
3. The GUI reopens the last project automatically. To start again, click **New project**.

> **Tip** The pretrained networks are taken from the repository `models` folder, or downloaded automatically from Zenodo the first time they are needed.

3. The window at a glance

The header holds the project buttons (**New project**, **Open project…**, **Load settings…**, **Save project**) and the **Theme** menu. Below are five tabs, used from left to right:

Tab	What you do there
1. Set up	Choose what to analyse, the input and results folders, the pixel size and time between frames; tick the videos to analyse.
2. Sample info	Give every video its condition (e.g. drug) and optionally genotype, patient / batch and marker.
3. Run pipeline	Tick the steps, click **Run**, follow the progress of every video and the log.
4. Check segmentation	Step through each video with the outlines drawn on top, to check that the objects are found and outlined well.
5. Analysis & results	Choose how to compare the objects, run the analysis and browse every figure and table.

Hover over any label, box or button to see a short explanation.

4. Tutorial A: organoid videos

This tutorial uses the folder `data/full_dataset_with_biomarker/26-SPOT-Drug screen-dye/Hoescht Only/Videos` of the repository: 15 brightfield videos of organoids treated with DMSO, cisplatin (6 and 100 µM), 5-fluorouracil or docetaxel. To keep it quick, only one video (`E5 5FU Hoechst 1`) is ticked for analysis; the steps are the same for all of them.

4.1 Set up (tab 1)

1. Under **What to analyse**, keep **Videos (SPOT finds, tracks and outlines the organoids)**.
2. **Videos folder**: click **Browse...** and choose the folder with your videos. The videos are listed on the right.
3. **Results folder**: choose a new, empty folder. Every result and the project file `spot_project.json` are written here; your videos are never changed.
4. **Dataset name** (optional): a short name used for the compiled tables.
5. **File types**: the video extensions to look for (`.avi .tif .tiff .wmv .mp4 .mov`).
6. If every brightfield video `<name> BF.AVI` has a fluorescent marker video `<name>.AVI`, tick **Each brightfield video has a matching fluorescent marker video** and check the name ending and marker channel. The organoids are found in the brightfield video and the marker is measured inside them.
7. **Pixel size** and **Time between frames**: as in your microscope metadata (the pixel size of the files as they are; resizing is corrected for automatically).
8. Keep **Resize frames to 512 x 512** and **Correct stage drift** ticked. Untick **Organoids are darker than the background** only for fluorescence videos with bright organoids.
9. Tick the videos to analyse in the **Use** column (all by default) and click **Save project**.

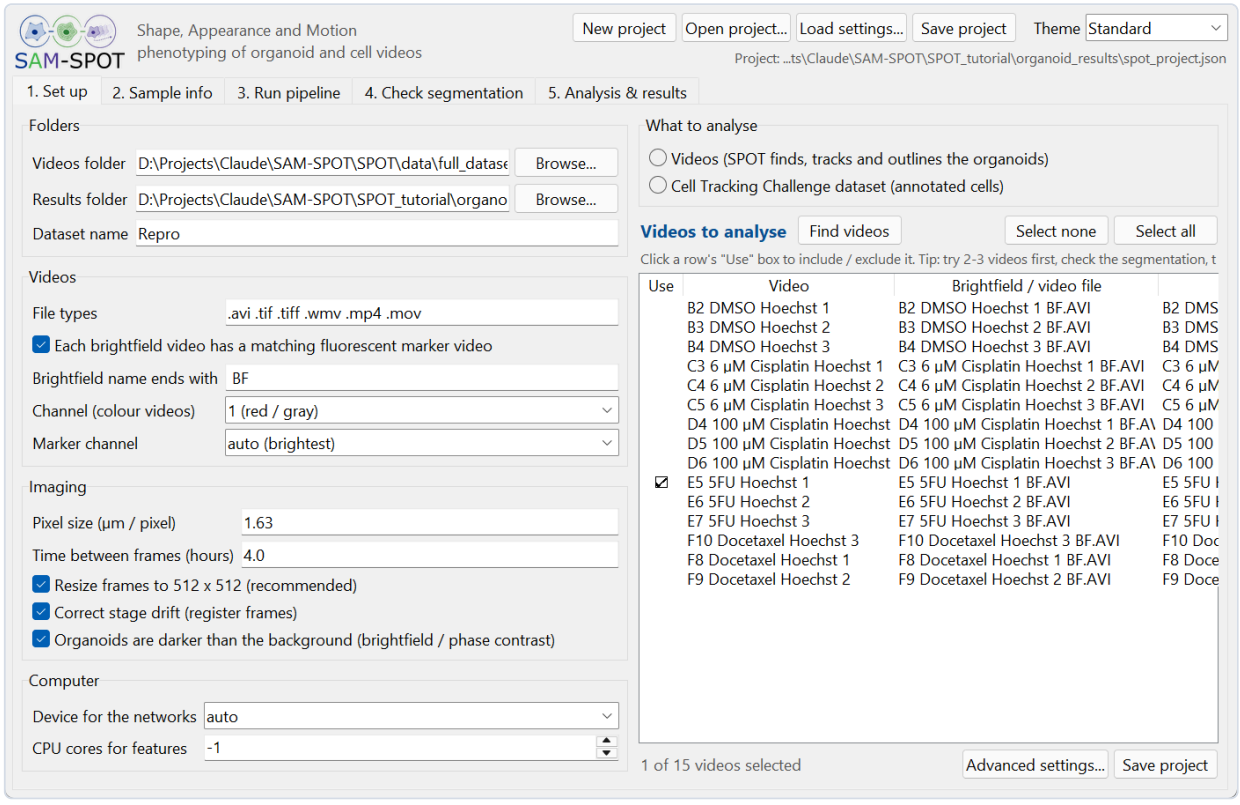


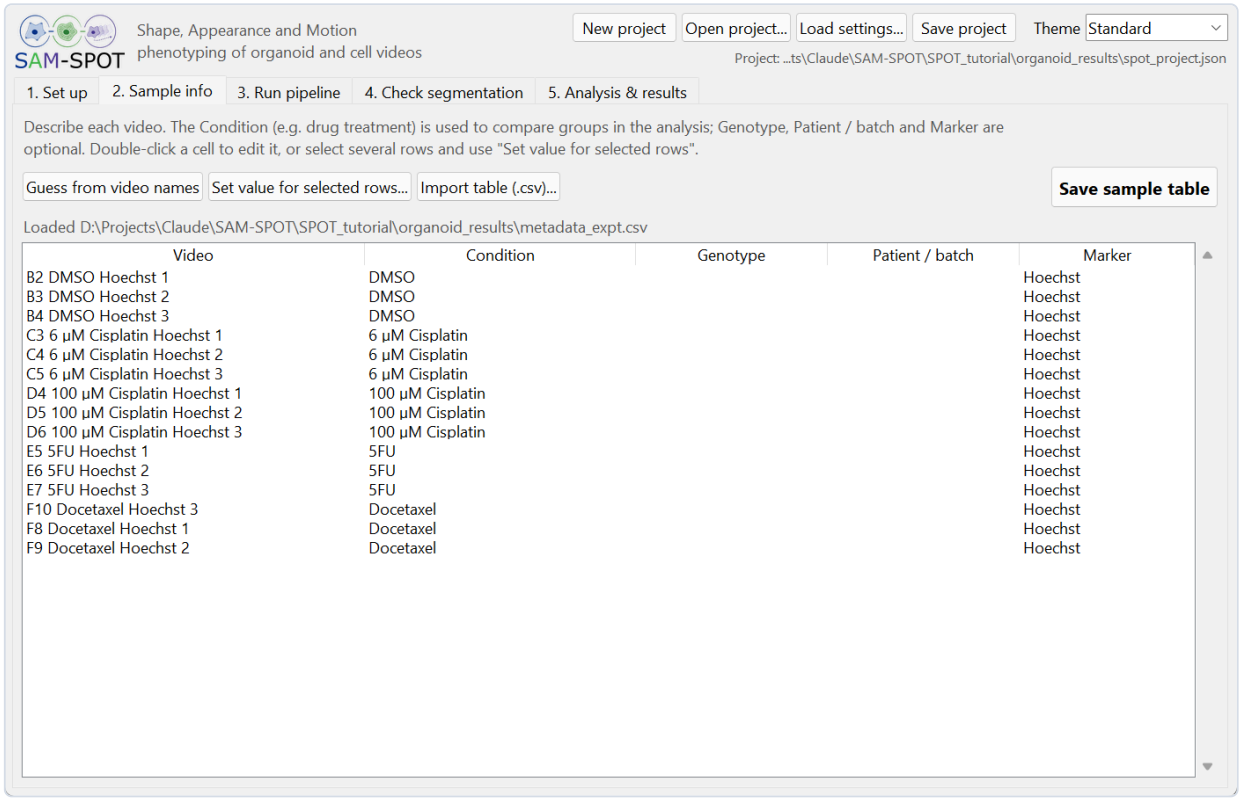
Figure 1. Tab 1 with the organoid video selected.

> **Tip** Start with 2 - 3 videos: run Stage 1, check the outlines on tab 4, adjust **Advanced settings...** if needed, then run all videos.

4.2 Sample information (tab 2)

Each row is a video. The **Condition** (e.g. the drug and dose) is what the analysis compares; **Genotype**, **Patient / batch** and **Marker** are optional.

- **Guess from video names** fills empty conditions from names such as `C10 6 µM Cisplatin Cytotox 2` (the well `C10` and replicate `2` are dropped; with marker videos the last word is the marker). Always check the result.
- Double-click a cell to edit it, or select several rows (Ctrl / Shift + click) and use **Set value for selected rows...**.
- **Import table (.csv)...** reads a table with the video name in the first column and columns `Condition`, `Genotype`, `Patient`, `Marker`.
- Click **Save sample table** (it is saved to `metadata_expt.csv` in the results folder).



Video	Condition	Genotype	Patient / batch	Marker
B2 DMSO Hoechst 1	DMSO			Hoechst
B3 DMSO Hoechst 2	DMSO			Hoechst
B4 DMSO Hoechst 3	DMSO			Hoechst
C3 6 µM Cisplatin Hoechst 1	6 µM Cisplatin			Hoechst
C4 6 µM Cisplatin Hoechst 2	6 µM Cisplatin			Hoechst
C5 6 µM Cisplatin Hoechst 3	6 µM Cisplatin			Hoechst
D4 100 µM Cisplatin Hoechst 1	100 µM Cisplatin			Hoechst
D5 100 µM Cisplatin Hoechst 2	100 µM Cisplatin			Hoechst
D6 100 µM Cisplatin Hoechst 3	100 µM Cisplatin			Hoechst
E5 5FU Hoechst 1	5FU			Hoechst
E6 5FU Hoechst 2	5FU			Hoechst
E7 5FU Hoechst 3	5FU			Hoechst
F10 Docetaxel Hoechst 3	Docetaxel			Hoechst
F8 Docetaxel Hoechst 1	Docetaxel			Hoechst
F9 Docetaxel Hoechst 2	Docetaxel			Hoechst

Figure 2. Tab 2: the condition of each video.

4.3 Run the pipeline (tab 3)

1. Tick the steps to run: all of them for a complete run, or **Stage 1 only** to check the segmentation first.
2. Keep **Skip steps already completed (resume)** ticked: finished steps are not recomputed, so you can stop a run and continue later.
3. Click **Run selected steps**. The table shows the status of every step of every video; the log below gives the details.

Symbol	Meaning
✓	done
·	to do
▶ running	being computed now
0 org.	no organoids were found in this video (it is left out of the analysis)
⟳	settings changed since it was computed: it is recomputed on the next run, even with *Skip steps already completed* ticked
✗ failed	an error occurred: the red lines of the log say why; the other videos continue

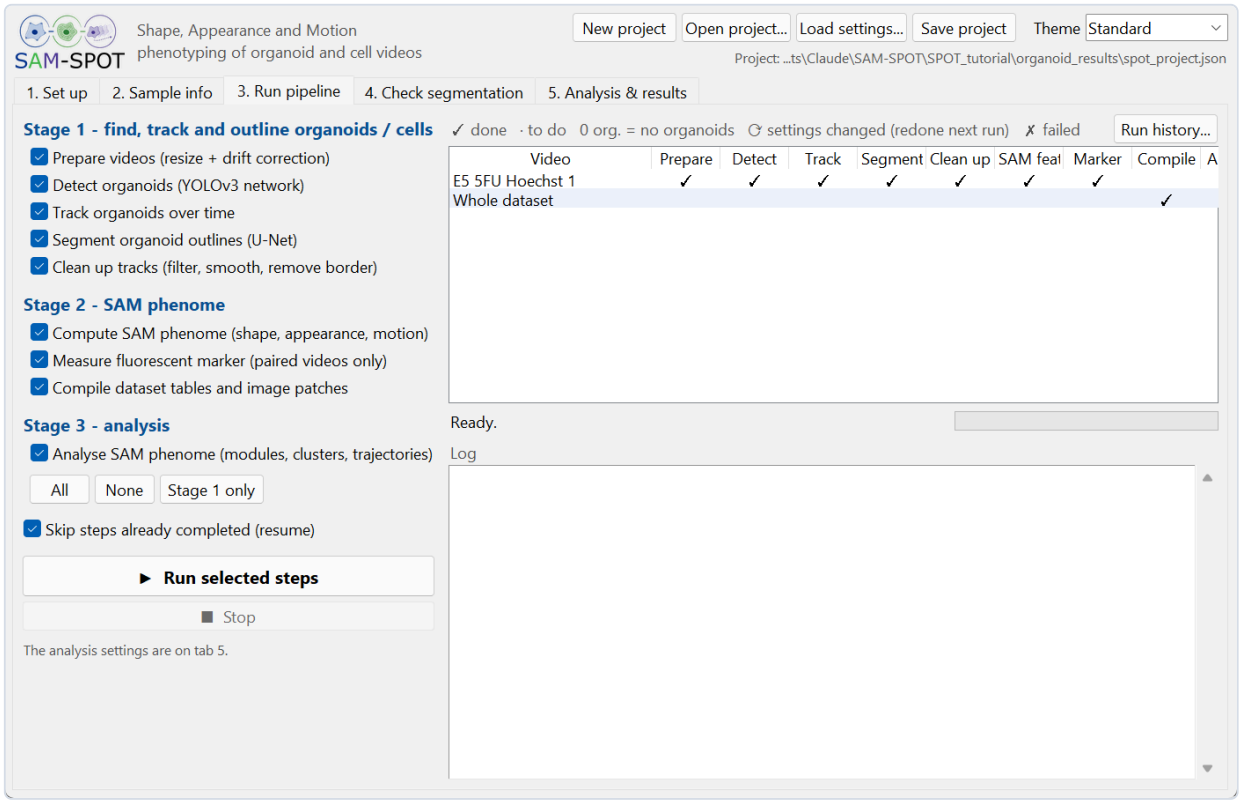


Figure 3. Tab 3 after a complete run of all steps.

As a guide, for the 1-video example on a CPU: preparing took under a minute, detection about 3 minutes (much less on a GPU), tracking, segmentation and clean-up under half a minute each, and the SAM features a few minutes. **Stop** ends a run; the step running at that moment is lost, finished steps are kept.

4.4 Check the segmentation (tab 4)

Choose a video and step through the frames with the slider, ◀ / ▶ or **Play**. Three layers can be drawn on top:

- **Final tracks**: the outlines used in the analysis, coloured and numbered by track;
- **Raw segmentation**: the U-Net outlines before clean-up;
- **Detected boxes**: the candidate boxes of the detector (brighter = more confident).

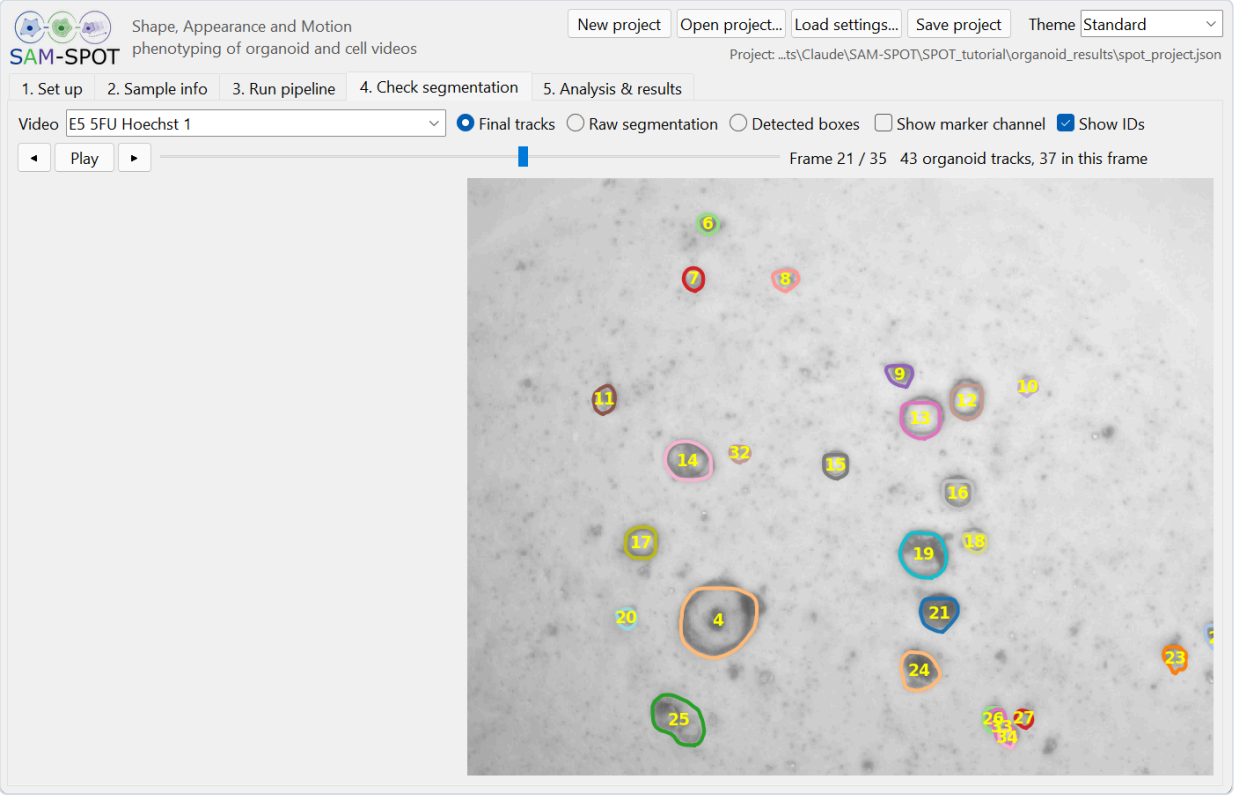


Figure 4. Tab 4: final organoid outlines on a frame of the video.

> **If organoids are missed or badly outlined** open **Advanced settings...** on tab 1. Lower the *Segmentation threshold* for larger outlines; lower *Min. track score* / *Min. track lifetime* to keep more organoids; increase the *Border width* to drop more organoids cut by the image edge. Then run again with *Skip steps already completed* ticked: only the steps affected by the changed settings (marked ⟳) are recomputed.

4.5 Analysis and results (tab 5)

1. **Analysis name**: results go to `analysis/<name>`; use a new name to keep an earlier analysis.
2. **Group by**: the sample information compared in the plots (usually *Condition*). **Batch** (optional): a column whose effect is regressed out, e.g. plate or experiment.
3. **Phenotype map and clusters**: the map method (UMAP or PaCMAP), the number of clusters (*auto* or a number), the SAM module scale and the number of time bins.
4. **Options**: keep the defaults (equal weighting of shape, appearance and motion; time-varying features only; HMM transitions). Tick *Also save figures as .svg* for publication figures.
5. Click **Run analysis**. It compiles the dataset first if needed. When it finishes, the results open on this tab: click a figure on the left to show it, double-click to open it (or a `.csv` table) in its own program.

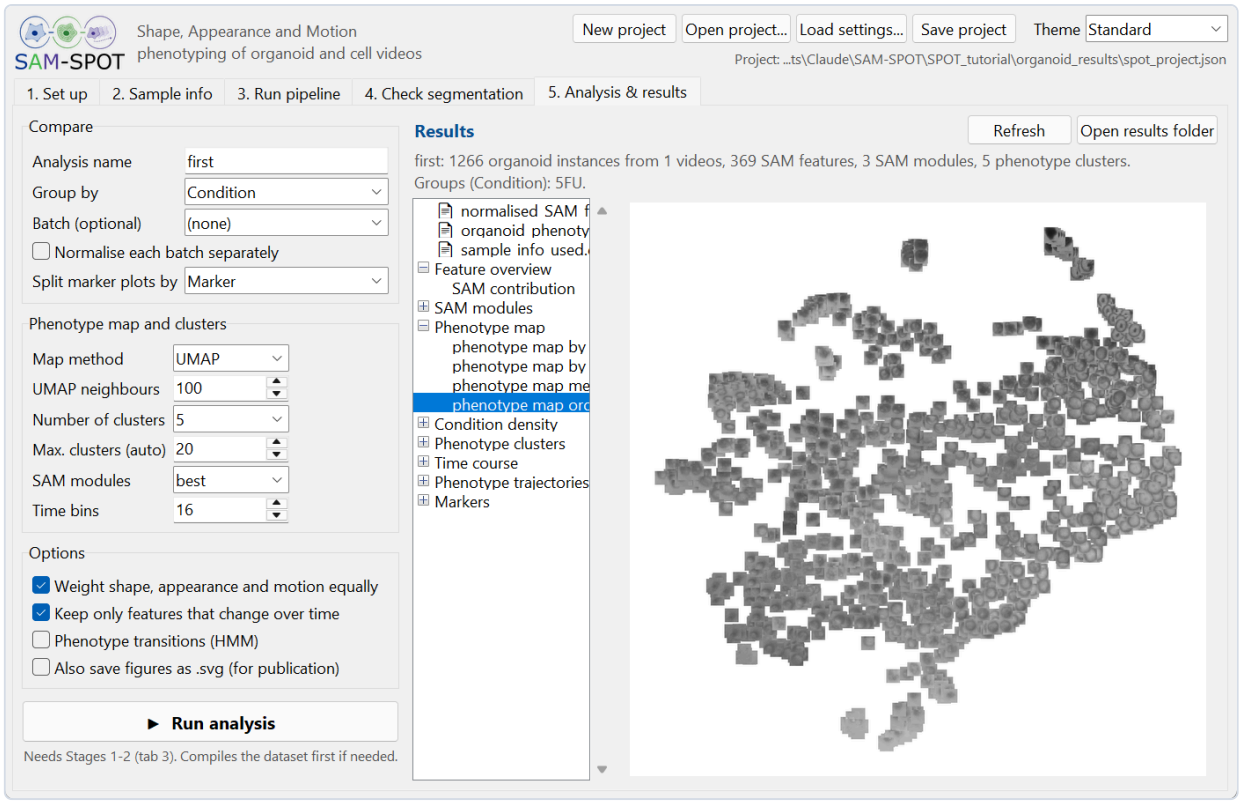


Figure 5. Tab 5: the analysis settings (left) and the results browser (right) showing the phenotype map with organoid images.

The line above the results summarises the analysis (number of organoid instances, features, modules and clusters). If a panel could not be made (e.g. too few organoids for the transitions), it is listed as *Not completed* and the rest of the analysis is still saved.

5. Tutorial B: a Cell Tracking Challenge dataset

The Cell Tracking Challenge provides 2D and 3D time-lapse videos of cells with reference segmentations and tracks. SPOT can phenotype the cells of its 2D datasets, using the annotations instead of the networks. The same option analyses the results of **your own** segmentation and tracking, if you save them in the challenge format.

5.1 Get a dataset

Download a 2D training dataset, e.g. **PhC-C2DH-U373** (glioblastoma-astrocytoma cells on a polyacrylamide substrate, phase contrast, 2 sequences of 115 frames), from `http://data.celltrackingchallenge.net/training-datasets/PhC-C2DH-U373.zip`, and unzip it. A dataset folder looks like this:

```
PhC-C2DH-U373/
  01/         t000.tif, t001.tif, ...          the frames of sequence 01
  01_GT/TRA/  man_track000.tif, ..., man_track.txt   gold truth tracks (a marker per cell)
  01_GT/SEG/  man_seg###.tif                    gold truth segmentation (some frames)
  01_ST/SEG/  man_seg###.tif                    silver truth segmentation (all frames)
  02/  02_GT/  02_ST/  ...                      sequence 02
  (01_RES/    mask###.tif, res_track.txt        your own results, optional)
```

5.2 Set up

1. Under **What to analyse**, choose **Cell Tracking Challenge dataset (annotated cells)**. The video options are replaced by the Cell Tracking Challenge options, and the resize / drift options are switched off (the frames must stay aligned with the masks).
2. **Dataset folder**: choose the dataset folder (`PhC-C2DH-U373`), or a folder containing several datasets. Every sequence is listed as `<dataset>-<sequence>`, with the track table it will use.
3. **Cell outlines from**:
   - **Best annotation** (recommended): for every tracked cell the silver truth segmentation, else the gold truth segmentation, else the gold truth tracking marker. Segmentation regions are matched to the tracks by their overlap with the tracking markers.
   - **Gold truth tracking markers only**: the `TRA` masks; for many datasets these are only small markers inside each cell, not its outline.
   - **My results**: the masks and `res_track.txt` in `<sequence>_RES`, e.g. from your own tracker.
4. **Min. track length**: shorter tracks are not analysed (default 5 frames).
5. Enter the **Pixel size** and **Time between frames** of the dataset. For PhC-C2DH-U373: **0.65 µm** per pixel and **15 minutes = 0.25 hours**.

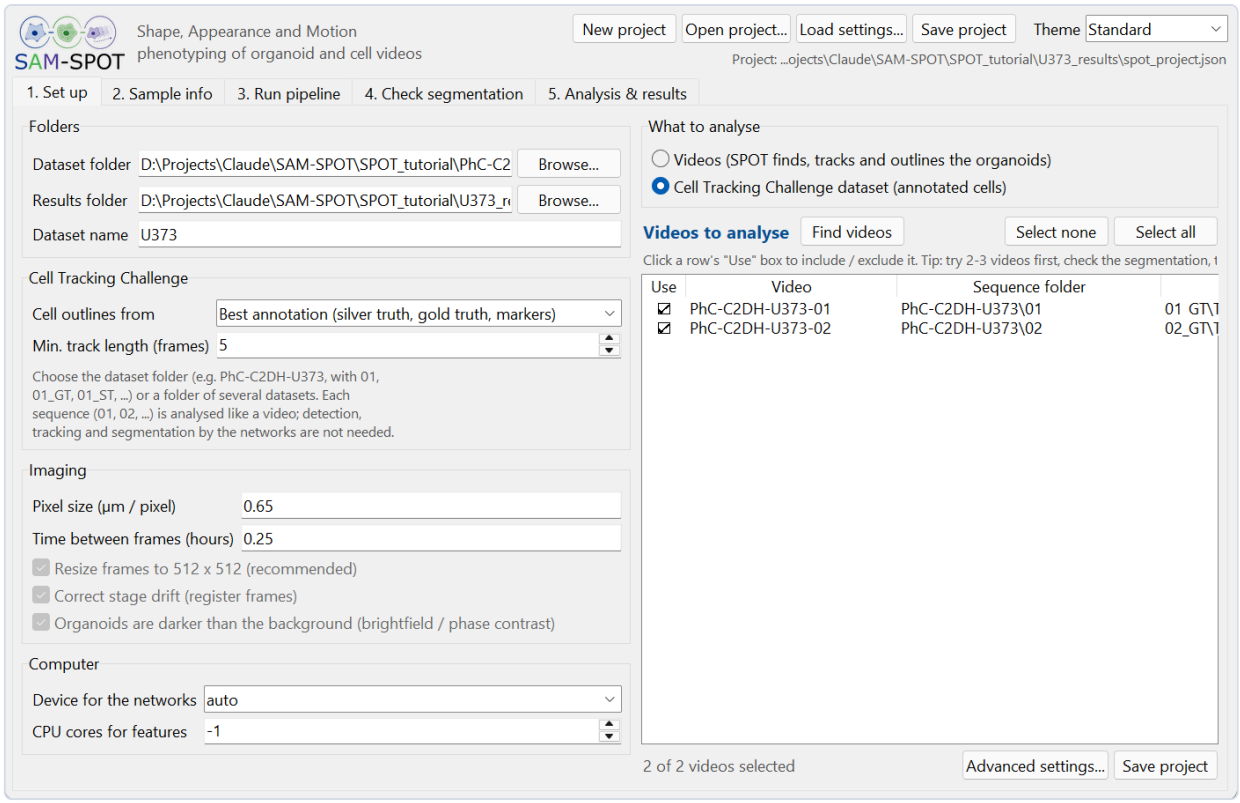


Figure 6. Tab 1 set up for the PhC-C2DH-U373 dataset: both sequences found with their gold truth track tables.

5.3 Sample information and run

On tab 2, **Guess from video names** sets the condition of every sequence to its dataset name (here `PhC-C2DH-U373`); with several datasets, compare them by *Condition*. On tab 3, **Detect**, **Track** and **Segment** are greyed out: they are not needed. The clean-up step is now **Outline the annotated cells, remove cells at the border** (column *Outlines*). Click **Run selected steps**.

- **Prepare** reads the frames as they are; 16-bit images are scaled to 8 bit between the 0.1 and 99.9 percentiles of the sequence.
- **Outlines** outlines every annotated cell of every track with 200 points, then removes cells touching the image border (*Border width / fraction* in Advanced settings) and tracks shorter than the minimum length. A track ends at a cell division: the daughter cells are new tracks, as in the challenge format.
- The SAM phenome, compilation and analysis are exactly as for organoids.

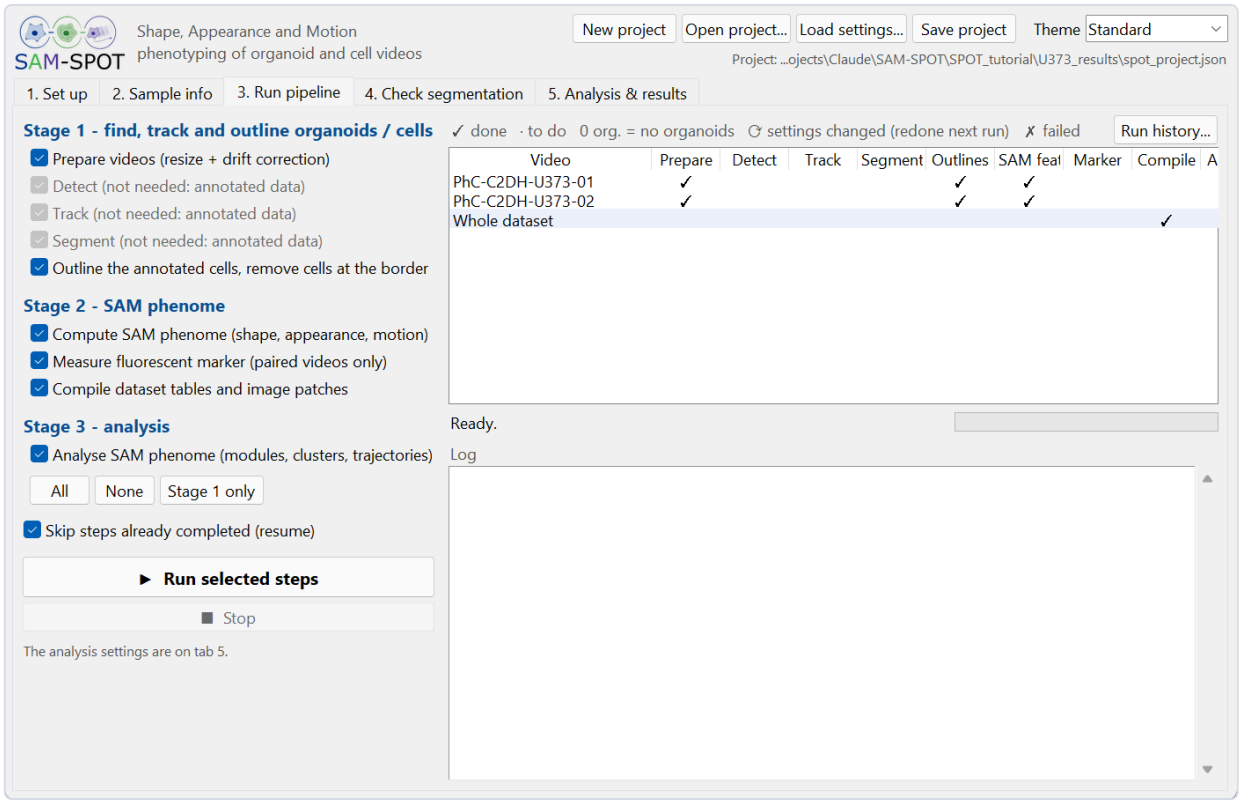


Figure 7. Tab 3 for the Cell Tracking Challenge dataset: detection, tracking and segmentation are not needed.

For PhC-C2DH-U373 the whole run took about a minute and a half on a 32-core workstation: 7 and 8 cell tracks are kept in sequences 01 and 02, giving 1074 cell instances.

5.4 Check the outlines and results

On tab 4, **Final tracks** shows the outlines that are analysed; **Raw segmentation** shows every annotated outline before the cells at the border were removed.

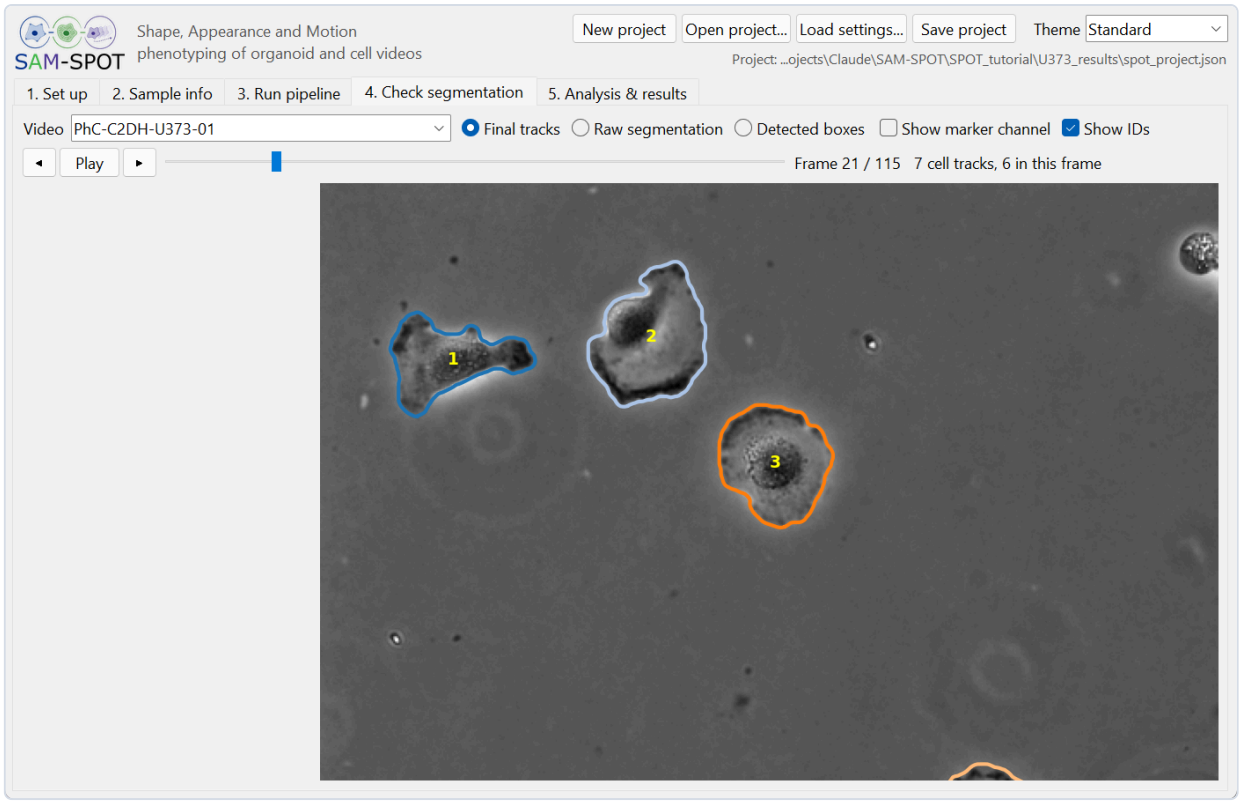


Figure 8. Tab 4: outlines of the annotated U373 cells (silver truth segmentation matched to the gold truth tracks).

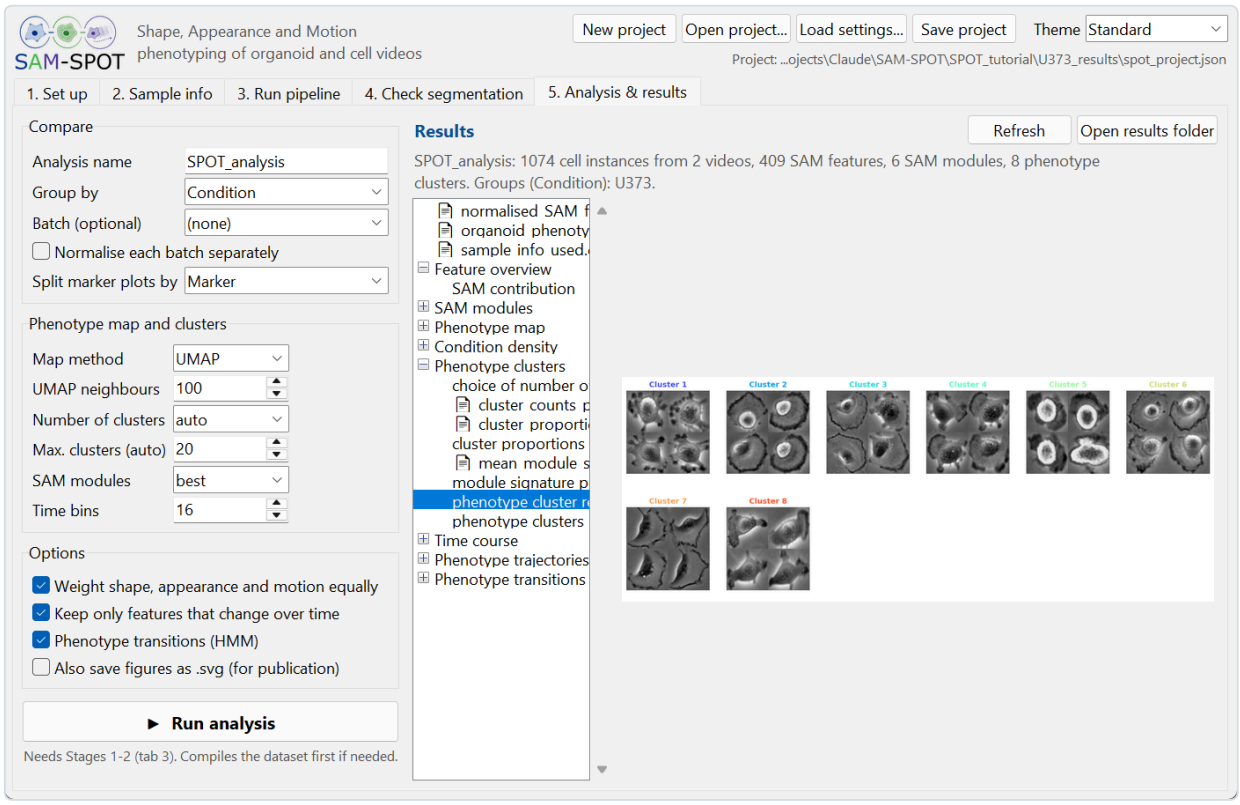


Figure 9. Tab 5: representative cells of each phenotype cluster of the U373 dataset.

6. Reading the results

Everything is saved in the results folder. The analysis folder `analysis/<name>` contains:

Folder / file	Contents
`00_feature_overview`	How much shape, appearance and motion each contribute.
`01_SAM_modules`	SAM modules: groups of co-varying features, their feature contributions, module scores on the map and the most representative objects of each module.
`02_phenotype_map`	The 2D phenotype map of all object instances coloured by group, time and key features, and with the object images.
`03_condition_density`	Where each group lies on the map.
`04_phenotype_clusters`	Phenotype clusters (ordered by elongation), the choice of their number, representative objects, module signature and proportions per group.
`05_time_course`	Cluster composition over time for each group.
`06_phenotype_trajectories`	The average phenotype trajectory of each group on the map and their similarity.
`07_phenotype_transitions`	Hidden Markov model transition matrix and graph between clusters, and the transition entropy per group.
`08_markers`	Marker measurements on the map and per group / cluster (paired marker videos only).
`organoid_phenotypes.csv`	One row per object and time point: sample information, map coordinates, cluster, module scores and marker values. Use it for your own statistics.
`normalised_SAM_features.csv`	The normalised SAM features used in the analysis.

The results folder also holds the compiled dataset tables `<dataset>_{shape,image,motion,marker}_features.csv`, the object outlines and image patches (`.mat`), and one folder per video under `videos/` with the intermediate results of every step.

7. Reproducibility, resuming and the command line

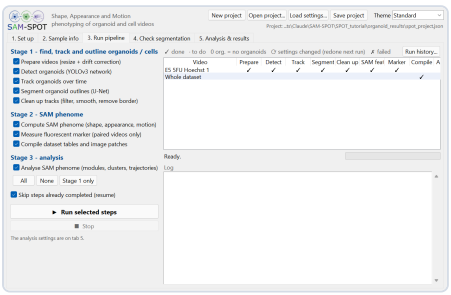
- **Projects**: the settings are saved as `spot_project.json` in the results folder. **Open project...** reopens it, also after the results folder was moved.
- **Run history**: every run writes `run_history/<date-time>/` with the complete settings (`run_config.json`), the sample table, the log and the versions of SPOT, Python, the main packages, the GPU and the network weights. **Run history...** on tab 3 opens it.
- **Load settings...** takes a `spot_project.json`, a `run_config.json` or an `analysis_config.json`, either reopening its folders or applying its settings to the current project, and can restore its sample table. Results computed with other settings are then marked ⟳.
- **Command line**: the same pipeline runs without the GUI, e.g. on a server:

```
python gui/spot_pipeline.py --config <results folder>/spot_project.json --steps
prepare,detect,track,segment,postprocess,features,markers,compile,analysis
```

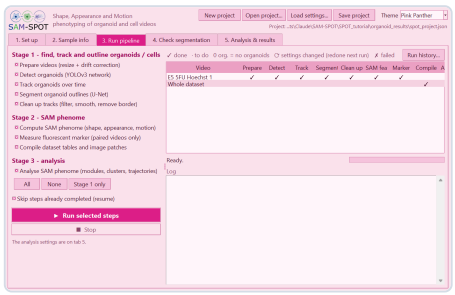
Steps that do not apply (e.g. detection for a Cell Tracking Challenge dataset) are skipped.

8. Themes

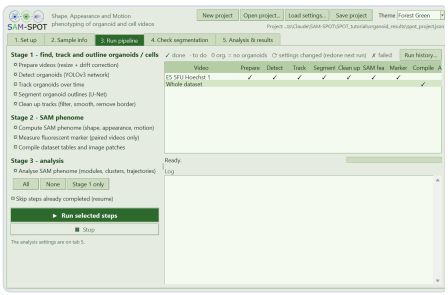
The **Theme** menu at the top right changes the colours of the window: **Standard** (the system look), **Pink Panther** and **Forest Green**. The choice is remembered for the next session. Themes change only the window, not the result figures.



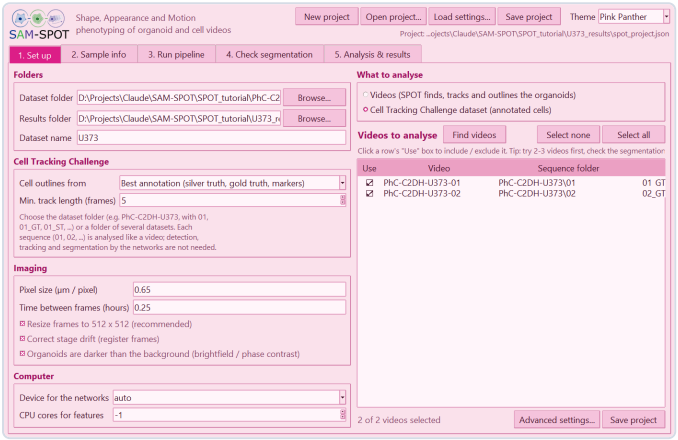
Standard



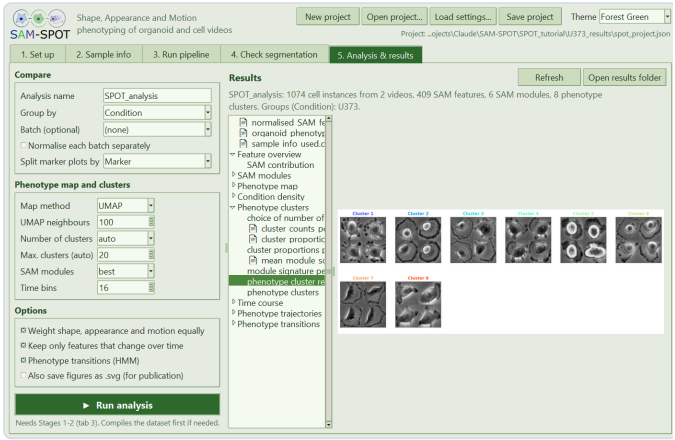
Pink Panther



Forest Green



Pink Panther: setting up a Cell Tracking Challenge dataset.



Forest Green: the results browser.

9. Troubleshooting

Problem	What to do
No videos found	Check the *File types*; with marker videos, check that the brightfield names end with the given text (e.g. `BF`). For a Cell Tracking Challenge dataset, choose the dataset folder that contains `01`, `02`, ...
"(no track table!)" next to a sequence	The sequence has no `<seq>_GT/TRA/man_track.txt` (or no `<seq>_RES/res_track.txt` with *My results*). Test datasets of the challenge come without annotations and cannot be used.
A step shows *X* failed	Read the red lines of the log on tab 3. The full log is in `run_history/<date-time>/log.txt`.
"the previous step has not been run"	Each step needs the one before it: tick the earlier steps too.
A video shows 0 org.	No organoid was left after clean-up. Check it on tab 4 (*Detected boxes*, *Raw segmentation*) and adjust the Advanced settings.
Not completed: ... on tab 5	One panel of the analysis could not be made, usually because there are too few objects or time points for it; the others are saved. The reason is in the log.
The networks are slow	Set *Device for the networks* to `cuda` if you have an NVIDIA GPU with a CUDA build of PyTorch.